

1주차(컴퓨터 구조)

≡ 멘토 이현재 멘토

<수업내용 요약>

1. 깃 허브 세팅(리눅스 개발환경 세팅)
2. 데이터 표현 실습(오버 플로우 재현,비트 끄기 실습)
3. 컴파일 과정 실습(gcc 컴파일러)

<과제>

1. 깃허브 코드스페이스로 리눅스 개발환경 만들기

▼ 코드스페이스는 GitHub 저장소 위에 떠 있는 클라우드 리눅스(Ubuntu) 컨테이너 + 브라우저 VS Code 이다.

만드는 순서:

1. 사용할 저장소(예: 과제용 repo)로 들어간다.
2. 초록색 `< > Code` 버튼 클릭 → **Codespaces** 탭 → **Create codespace on main** 클릭.
3. 브라우저 안에 VS Code가 열리는데, 이게 바로 리눅스 개발환경 이다.

2. 오버플로 예제를 언더플로로 바꿔서 해보기 CHAR_MIN의 값에서 -1

```
#include <stdio.h>
#include <limits.h>

int main() {
    char value = CHAR_MIN;
    printf("Original value : %d\n", value);
    value = value - 1;
    printf("Value after subtracting 1 : %d\n", value);
    return 0;
}
```

```
[Running] cd "d:\이승환\명지대학교\화이트햇스쿨\1주차\컴퓨터구조\" && gcc char_underflow.c -o char_underflow &&
"d:\이승환\명지대학교\화이트햇스쿨\1주차\컴퓨터구조\"char_underflow
CHAR_MIN      = -128
CHAR_MIN - 1 = 127
```

기존 실습 코드의 value 부분을 CHAR_MIN으로 바꾸고 value-1로 바꾸었다.

c언어 char자료형의 가장 작은 값은 -128이고 이진수로 표현하면 10000000가 된다.

여기에 1을 빼주면 원래는 -129가 되어야 하지만 더 작아질 수 없기 때문에
 $10000000 - 1 = 01111111$ 이 되어 127이라는 값이 나오게 된다.

3. 비트 연산 프로그램 바꿔보기 특정위치의 비트를 끄는 함수 구현

- 사용자의 입력 (특정위치 -int)값을 받도록 수정

```
#include <stdio.h>

//특정위치의 비트를 끄는 함수
//끄고자 하는 위치의 비트를 1로바꿈
//1<<(n) : n번째 비트 위치에 1을 만들어냄
//~연산으로 모든 비트를 반전시킴
//그 상태에서 & 연산을 하면 특정 위치의 비트만 0이 되고 나머지는 그
대로 유지됨

//특정 위치의 비트 값이 1인지 확인하는 함수
int is_bit_set(unsigned char value, int bit_position) {
    return (value & (1 << bit_position)) != 0;
}

//특정 위치의 비트 값을 1로 설정하는 함수
unsigned char set_bit(unsigned char value, int bit_positio
n) {
    return value | (1 << bit_position);
}

//특정 위치의 비트를 끄는(0으로 만드는) 함수
unsigned char clear_bit(unsigned char value, int bit_positi
on) {
```

```

        return value & ~(1 << bit_position);
    }

//unsigned char 값을 8자리 2진수 문자열로 변환하는 함수
//MSB(7번 비트)부터 LSB(0번 비트)까지 순서대로 채운다.
void to_binary(unsigned char value, char *buffer) {
    for (int i = 7; i >= 0; i--) {
        buffer[7 - i] = (value & (1 << i)) ? '1' : '0';
    }
    buffer[8] = '\0'; // 문자열 끝을 표시
}

int main(){
    unsigned char value = 0b1111111; // 초기값: 모든 비트가
1인 상태
    int bit_position;                // 사용자가 끌 비트 위
치
    char bin[9];                    // 2진수 표현을 담을
버퍼 (8비트 + 널문자)

    to_binary(value, bin);
    printf("Initial value: %u (binary: %s)\n", value, bin);

    // 끄고자 하는 비트 위치를 사용자로부터 입력받는다. (0 ~ 7)
    printf("Enter the bit position to clear (0-7): ");
    if (scanf("%d", &bit_position) != 1) {
        printf("Invalid input.\n");
        return 1;
    }

    // 입력값이 유효한 범위(0~7)인지 검사한다.
    if (bit_position < 0 || bit_position > 7) {
        printf("Bit position must be between 0 and 7.\n");
        return 1;
    }

    unsigned char original = value; // 원래값을 보관해 둔다.

```

```

// 입력받은 위치의 비트를 끈다.
value = clear_bit(value, bit_position);

// 원래값과 끈 값을 10진수와 2진수로 함께 출력한다.
to_binary(original, bin);
printf("Original value : %3u (binary: %s)\n", original,
bin);

to_binary(value, bin);
printf("Cleared value : %3u (binary: %s)\n", value, bin);

return 0;
}

```

```

cd "d:\이승환\명지대학교\화이트햇스쿨\1주차\컴퓨터구조\";
Initial value: 255
Enter the bit position to clear (0-7): 3
Value after clearing bit 3: 247
PS D:\이승환\명지대학교\화이트햇스쿨\1주차\컴퓨터구조>

```

처음 비트 끄기 코드를 생성하였을때는 비트가 꺼진 후 정수형으로 변환된 수가 출력되는데 이렇게 출력되면 어느 위치의 비트가 꺼지게 된 것 인지 알 수가 없어서 이진 수 값도 표시해주는 코드를 작성하였다.

```

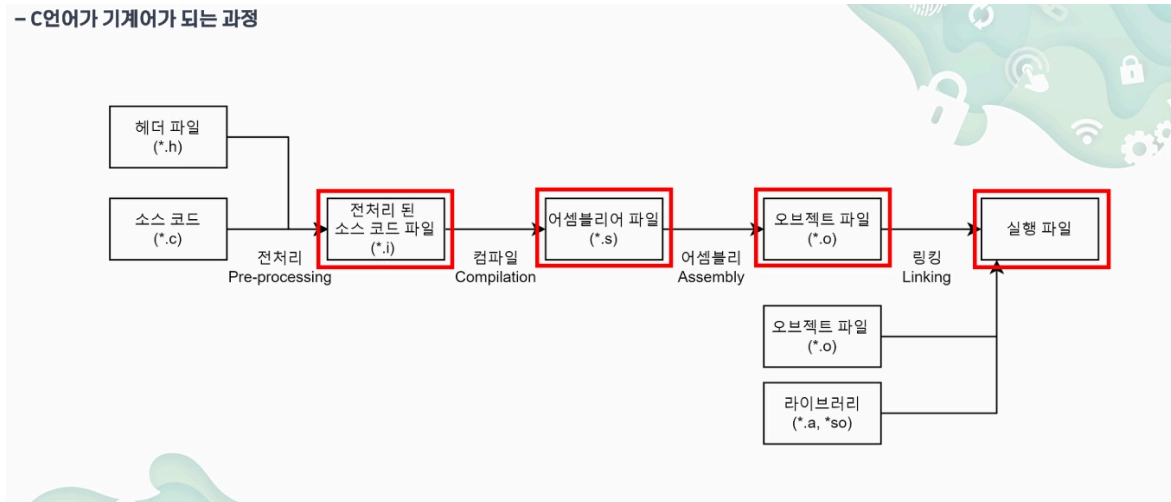
cd "d:
Initial value: 255 (binary: 11111111)
Enter the bit position to clear (0-7): 3
Original value : 255 (binary: 11111111)
Cleared value : 247 (binary: 11110111)
PS D:\이승환\명지대학교\화이트햇스쿨\1주차\컴퓨터구조>

```

0-7까지의 총 8자리 이진수 에서 3 즉 4번째 자리의 비트가 꺼지게 되어서 해당 값이 출력되는것을 확인할 수 있다.

4. c언어가 기계어가 되는 과정 직접 해보기

- C언어가 기계어가 되는 과정



c언어가 기계어가 되는 과정을 수업시간에서 사용한 hello_world.c 코드를 수정해보았다.
gcc -E hello_world.c -o hello_world.i 를 터미널에 입력하여 전처리를 한다.

```

# 1 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/corecrt_stdio_
# 10 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/corecrt_stdio_
# 1 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/corecrt.h" 1
# 10 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/corecrt.h" 3
# 1 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/_mingw.h" 1
# 10 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/_mingw.h" 3
# 1 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/_mingw_mac.h"
# 108 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/_mingw_mac.h"
# 117 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/_mingw_mac.h"
# 326 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/_mingw_mac.h"
# 405 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/_mingw_mac.h"
# 11 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/_mingw.h" 2
# 1 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/_mingw_secapi.h"
# 12 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/_mingw.h" 2
# 306 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/_mingw.h" 3
# 1 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/vadefs.h" 1
# 9 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/vadefs.h" 3
# 1 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/_mingw.h" 1
# 10 "C:/Users/samsung/AppData/Local/Microsoft/WinGet/Packages/BrechtSanders.WinLibs.POSIX.UCRT_Microsoft.Winget.Source_8wekyb3d8bbwe/mingw64/x86_64-mingw32/include/vadefs.h" 2

```

사진과 같이 전처리된 알수 없는 값들이 입력된 파일이 생성된다.

그 뒤에 gcc -S hello_world.i -o hello_world.s 명령어를 통해 어셈블리어로 변환된 코드를 볼 수 있다.

```

1주차 > 컴퓨터구조 > ASM hello_world.s
1  .file "hello_world.c"
2  .text
3  .section .rdata,"dr"
4  .LC0:
5  .ascii "Hello, World!\0"
6  .text
7  .globl main
8  .def main; .scl 2; .type 32; .endef
9  .seh_proc main
10 main:
11 pushq %rbp
12 .seh_pushreg %rbp
13 movq %rsp, %rbp
14 .seh_setframe %rbp, 0
15 subq $32, %rsp
16 .seh_stackalloc 32
17 .seh_endprologue
18 call __main
19 leaq .LC0(%rip), %rax
20 movq %rax, %rcx
21 call puts
22 movl $0, %eax
23 addq $32, %rsp
24 popq %rbp
25 ret
26 .seh_endproc
27 .def __main; .scl 2; .type 32; .endef
28 .ident "GCC: (MinGW-W64 x86_64-ucrt-posix-seh, built by Brecht Sanders, r2) 16.1.0"
29 .def puts; .scl 2; .type 32; .endef
30

```

```

● @tortoise → /workspaces/WHS-4TH-study/1주차/컴퓨터구조 (dev) $ file *
char_overflow.c: C source, Unicode text, UTF-8 text
char_underflow.c: C source, ASCII text
hello_world.c: C source, ASCII text
hello_world.i: C source, ASCII text
hello_world.s: assembler source, ASCII text
off_bit.c: C source, Unicode text, UTF-8 text
sizeof_tvdes.c: C source, Unicode text, UTF-8 text

```

file * 명령어를 통해 각 파일들이 어떻게 작성되었는지를 확인 할 수 있다.

이렇게 c언어가 기계어로 변하는 과정을 직접 실습하며 에셈블리어에대해서 좀더 깊게 공부 해 본다면 좋을 것 같다는 생각이 들었다.